

A Technical Introduction to Denoising diffusion probabilistic model

Section 1

"network", because f_{θ} is implemented as a neural network. "score", because the output of the network is interpreted as approximating the score function $\nabla \ln \rho_t$. "noise conditional", because ρ_t is equal to ρ_0 blurred by an added Gaussian noise that increases with time, and so the score function depends on the amount of noise added.

Section 2

$$dy_t = \frac{1}{2} \beta(T-t) y_t dt + \beta(T-t) \nabla y_t \ln \rho_{T-t}(y_t) dt + \beta(T-t) dW_t$$
 score function
$$dy_t = \frac{1}{2} \beta(T-t) y_t dt + \beta(T-t) \nabla y_t \ln \rho_{T-t}(y_t) dt + \beta(T-t) dW_t$$

Section 3

The idea of score functions Consider the problem of image generation. Let x represent an image, and let $q(x)$ be the probability distribution over all possible images. If we have $q(x)$ itself, then we can say for certain how likely a certain image is. However, this is intractable in general. Most often, we are uninterested in knowing the absolute probability of a certain image. Instead, we are usually only interested in knowing how likely a certain image is compared to its immediate neighbors — e.g. how much more likely is an image of cat compared to some small variants of it? Is it more likely if the image contains two whiskers, or three, or with some Gaussian noise added? Consequently, we are actually quite uninterested in $q(x)$ itself, but rather, $\nabla_x \ln q(x)$. This has two major effects:

Section 4

$$d\phi_t(x) = v_t(\phi_t(x)) dt, \text{ starting from } \phi_0(x) = x$$

Section 5

To construct a probability path, we start by construct a conditional probability path $p_t(x|z)$ and the corresponding conditional velocity field $v_t(x|z)$ on some conditional distribution $q(z)$. A natural choice is the Gaussian conditional probability path:

Section 6

Further reading Review papers Yang, Ling (2024-09-06), YangLing0818/Diffusion-Models-Papers-Survey-Taxonomy, retrieved 2024-09-06 Yang, Ling; Zhang, Zhilong; Song, Yang; Hong, Shenda; Xu, Runsheng; Zhao, Yue; Zhang, Wentao; Cui, Bin; Yang, Ming-Hsuan (2023-11-09). "Diffusion Models: A Comprehensive Survey of Methods and Applications". ACM Comput. Surv. 56 (4): 105:1–105:39. arXiv:2209.00796. doi:10.1145/3626235. ISSN 0360-0300. Austin, Jacob; Johnson, Daniel D.; Ho, Jonathan; Tarlow, Daniel; Rianne van den Berg (2021). "Structured Denoising Diffusion Models in Discrete State-Spaces". arXiv:2107.03006 [cs.LG]. Croitoru, Florinel-Alin; Hondru, Vlad; Ionescu, Radu Tudor; Shah, Mubarak (2023-09-01). "Diffusion Models in Vision: A Survey". IEEE Transactions on Pattern Analysis and Machine Intelligence. 45 (9): 10850–10869. arXiv:2209.04747. Bibcode:2023ITPAM..4510850C. doi:10.1109/TPAMI.2023.3261988. ISSN 0162-8828. PMID 37030794. Mathematical details omitted in the article. "Power of Diffusion Models". AstraBlog. 2022-09-25. Retrieved 2023-09-25. Luo, Calvin (2022-08-25). "Understanding Diffusion Models: A Unified Perspective". arXiv:2208.11970 [cs.LG]. Weng, Lilian (2021-07-11). "What are Diffusion Models?". lilianweng.github.io. Retrieved 2023-09-25. Tutorials Nakkiran, Preetum; Bradley, Arwen; Zhou, Hattie; Advani, Madhu (2024). "Step-by-Step Diffusion: An Elementary Tutorial". arXiv:2406.08929 [cs.LG]. "Guidance: a cheat code for diffusion models". 26 May 2022. Overview of classifier guidance and classifier-free guidance, light on mathematical details. Catherine Higham, Desmond J. Higham, and Peter Grindrod: "Diffusion Models for Generative Artificial Intelligence: An Introduction for Applied Mathematicians", SIAM Review, Vol.67, No.3 (2025).

Section 7

The DALL-E series by OpenAI are text-conditional diffusion models of images. The first version of DALL-E (2021) is not actually a diffusion model. Instead, it uses a Transformer architecture that autoregressively generates a sequence of tokens, which is then converted to an image by the decoder of a discrete VAE. Released with DALL-E was the CLIP classifier, which was used by DALL-E to rank generated images according to how close the image fits the text. GLIDE (2022–03) is a 3.5-billion diffusion model, and a small version was released publicly. Soon after, DALL-E 2 was released (2022–04). DALL-E 2 is a 3.5-billion cascaded diffusion model that generates images from text by "inverting the CLIP image encoder", the technique which they termed "unCLIP". The unCLIP method contains 4 models: a CLIP image encoder, a CLIP text encoder, an image decoder, and a "prior" model (which can be a diffusion model, or an autoregressive model). During training, the prior model is trained to convert CLIP image encodings to CLIP text encodings. The image decoder is

trained to convert CLIP image encodings back to images. During inference, a text is converted by the CLIP text encoder to a vector, then it is converted by the prior model to an image encoding, then it is converted by the image decoder to an image. Sora (2024-02) is a diffusion Transformer model (DiT).

Section 8

Forward diffusion process Consider again the forward diffusion process, but this time in continuous time: $x_t = \sqrt{1 - \beta_t} x_0 + \sqrt{\beta_t} z_t$ By taking the $\beta_t \rightarrow \beta(t) dt$, $dt z_t \rightarrow dW_t$ limit, we obtain a continuous diffusion process, in the form of a stochastic differential equation: $dx_t = -\frac{1}{2} \beta(t) x_t dt + \sqrt{\beta(t)} dW_t$ where W_t is a Wiener process (multidimensional Brownian motion). Now, the equation is exactly a special case of the overdamped Langevin equation $dx_t = -\frac{D}{k_B T} (\nabla_x U) dt + \sqrt{2D} dW_t$ where D is diffusion tensor, T is temperature, and U is potential energy field. If we substitute in $D = \frac{1}{2} \beta(t) I$, $k_B T = 1$, $U = \frac{1}{2} \|x\|^2$, we recover the above equation. This explains why the phrase "Langevin dynamics" is sometimes used in diffusion models. Now the above equation is for the stochastic motion of a single particle. Suppose we have a cloud of particles distributed according to q at time $t = 0$, then after a long time, the cloud of particles would settle into the stable distribution of $N(0, I)$. Let ρ_t be the density of the cloud of particles at time t , then we have $\rho_0 = q$; $\rho_T \approx N(0, I)$ and the goal is to somehow reverse the process, so that we can start at the end and diffuse back to the beginning. By Fokker-Planck equation, the density of the cloud evolves according to $\partial_t \ln \rho_t = \frac{1}{2} \beta(t) (n + (x + \nabla \ln \rho_t) \cdot \nabla \ln \rho_t + \Delta \ln \rho_t)$ where n is the dimension of space, and Δ is the Laplace operator. Equivalently, $\partial_t \rho_t = \frac{1}{2} \beta(t) (\nabla \cdot (x \rho_t) + \Delta \rho_t)$